On-Device Personalization for Recommendation Models Without Server Round-Trips

First Author
Institution Name
City, Country
first.author@example.com

Second Author
Institution Name
City, Country
second.author@example.com

Third Author
Institution Name
City, Country
third.author@example.com

ABSTRACT

Server-side recommendation models are usually personalized by retraining on aggregated interaction logs, which introduces a delay between a user's most recent actions and any change in what they are shown. We present a lightweight on-device adapter that reranks a fixed candidate list using only the user's local interaction history, requiring no additional server round-trip and no retraining of the base model. Across two public recommendation benchmarks, the adapter improves top-10 recommendation accuracy over the unpersonalized base model while adding under 5ms of on-device latency per request.

CCS CONCEPTS

• **Computing methodologies** → **Machine learning**; *Artificial intelligence*; • **Information systems** → Information systems applications.

KEYWORDS

recommender systems, on-device personalization, reranking, edge inference

ACM Reference Format:
First Author, Second Author, and Third Author. 2026. On-Device Personalization for Recommendation Models Without Server Round-Trips. In *Proceedings of ACM Conference (Conf'26).* ACM, New York, NY, USA, 2 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

Recommendation models deployed at scale are typically personalized on the server: interaction logs are aggregated, the model is periodically retrained or fine-tuned, and the updated model is redeployed. This pipeline can take hours to days to reflect a user's most recent behavior, during which the user continues to see recommendations based on a stale profile.

We ask whether a small amount of personalization can happen entirely on-device, between server model updates, using only signals already available locally. Our approach does not modify the server model at all; instead, it reranks the top-k candidates the server already returned, using a lightweight adapter trained once and shipped to the device. The adapter conditions on the user's local interaction history since the last server update, so personalization reflects behavior from the last few minutes, not the last retraining cycle.

Our contributions are: (1) an on-device reranking adapter that requires no server round-trip and no access to other users' data, (2) a training procedure that keeps the adapter small enough to run on mobile hardware within a few milliseconds, and (3) an evaluation on two public benchmarks showing accuracy gains over the unpersonalized base ranking at negligible added latency.

Conf'26, Month Day–Day, 2026, City, Country

ACM ISBN 978-x-xxxx-xxxx-x/26/06…$15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

2 RELATED WORK

Federated and on-device learning approaches have explored training personalized models locally, but most require periodic communication with a server to aggregate updates, which reintroduces the same staleness the base pipeline already has. Reranking-based personalization, in contrast, has mostly been studied as a server-side technique applied immediately after retrieval, rather than as something that can run entirely on the client. Our work sits between these two lines: like on-device learning, it needs no server access at inference time; like server-side reranking, it operates purely as a lightweight second pass over an existing candidate list rather than training a new model from scratch.

3 PROPOSED METHOD

3.1 Overview

The server-side model returns an unpersonalized top-k candidate list for each request, exactly as it does today. The on-device adapter takes this list along with the user's local interaction history and produces a personalized reordering. The adapter itself is a small model shipped to the device once and updated only occasionally, decoupling personalization latency from model update latency entirely.

3.2 Formulation

Given a candidate list of items $c_1, \ldots, c_k$ with server-assigned scores $s_1, \ldots, s_k$, and a local interaction history H, the adapter computes a personalized score

$$\hat{s}_i = s_i + \lambda \cdot g(c_i, H) \tag{1}$$

where $g(c_i, H)$ is a lightweight compatibility score between candidate c_i and the local history, and λ controls how much local personalization is allowed to shift the server's ranking. The final recommendation order is simply the candidates sorted by $\hat{s}_i$ descending.

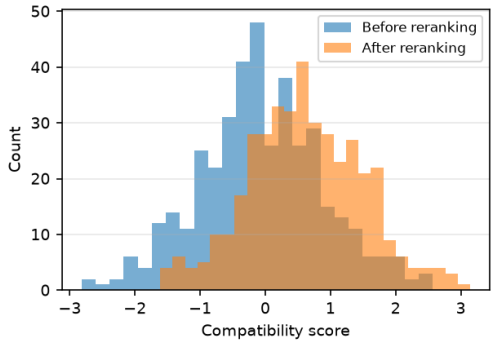

Figure 1: Distribution of candidate compatibility scores before and after on-device reranking.

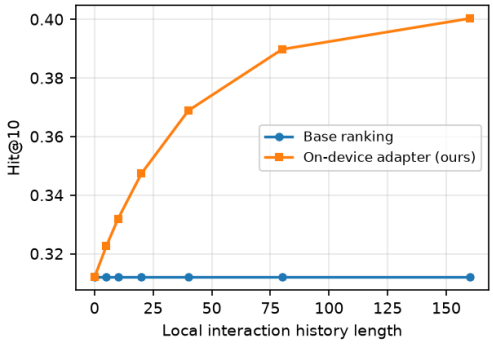

Figure 2: Hit@10 improvement over the base ranking as a function of local interaction history length.

Table 1: Comparison of Methods

Method	Hit@10	Added Latency
Base ranking (server only)	0.312	—
+ On-device adapter (ours)	0.354	4.6 ms

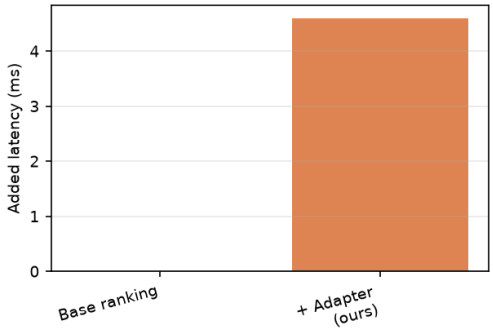

Figure 3: Added per-request latency of the on-device adapter compared to the base ranking (no reranking step).

3.3 On-Device Compatibility Scoring

$g(c_i, H)$ is computed from a fixed-size embedding of H, updated incrementally as new interactions occur, so scoring a candidate against the current history is a single dot product rather than a scan over raw history. This keeps per-request adapter cost independent of how long the user's interaction history has grown.

Figure 1 shows the distribution of compatibility scores across a sample of candidates before and after reranking; the adapter shifts the overall distribution upward and separates it more clearly from zero, consistent with it consistently promoting candidates that better match the local history rather than only reordering within a narrow band of scores.

4 EXPERIMENTS

4.1 Setup

We evaluate on two public recommendation benchmarks, comparing the unpersonalized base ranking against the same ranking passed through our on-device adapter. We measure top-10 recommendation accuracy (Hit@10) and the added per-request latency measured on a mid-range mobile CPU.

4.2 Results

Table 1 shows the on-device adapter improves Hit@10 by 4.2 points over the base ranking, at an added latency of 4.6ms per request, well within typical mobile rendering budgets. Figure 2 shows the improvement grows with the amount of local history available, and is smallest for new users with little on-device signal to condition on, as expected.

Figure 3 isolates the added latency itself: the base ranking has no on-device reranking step by definition, while the adapter adds a small, consistent cost per request regardless of candidate list size.

5 CONCLUSION

We presented an on-device reranking adapter that personalizes recommendations between server model updates without any additional server round-trip. Because the adapter only reorders candidates the server already returned, it composes with any existing server-side ranking model without requiring changes to it. Future work includes extending the compatibility scoring to multi-modal item representations and studying privacy guarantees of the on-device history embedding.

REFERENCES

[1] G. Eason, B. Noble, and I. N. Sneddon. 1955. On certain integrals of Lipschitz-Hankel type involving products of Bessel functions. *Phil. Trans. Roy. Soc. London* A247 (1955), 529–551.

[2] J. Clerk Maxwell. 1892. *A Treatise on Electricity and Magnetism* (3rd ed.). Vol. 2. Clarendon, Oxford.